📘 Military Asset Management System

🚀 Project Overview

The Military Asset Management System is a full-stack web application built using React, Node.js, and MongoDB to manage and track military assets such as vehicles, weapons, and ammunition across multiple bases.

The system enables users to:

Manage asset inventory

Track transfers between bases

Handle assignments and expenditures

Ensure secure access using Role-Based Access Control (RBAC)

🛠️ Tech Stack

Frontend

React.js

Axios

React Router

Backend

Node.js

Express.js

Database

MongoDB (Mongoose)

📂 Project Structure

---

military-asset-management/

|

├── backend/

| ├── models/

| ├── routes/

| ├── middleware/

| └── index.js

|

└── frontend/

├── src/

| ├── components/

| ├── api.js

| └── App.js

🔐 Features

✅ Authentication & Authorization

JWT-based authentication

Role-Based Access Control (RBAC)

👥 Roles

Admin → Full access

Commander → Transfers & Assignments

Logistics Officer → Purchases

📦 Core Functionalities

1. Asset Management (Purchases)

Add assets to specific bases

Track available stock

2. Transfers

Move assets between bases

Automatically updates stock:

Deducts from source base

Adds to destination base

3. Assignments & Expenditures

- ASSIGNED

Assets given to personnel

Example: 10 rifles assigned to a soldier

- EXPENDED

Assets consumed or unusable

Example: 5 bullets used in training

4. Dashboard

Displays all assets

Shows quantity and base

Includes filtering options

5. Popup Details

View transfer history

View assignment/expenditure history

⚙️ Setup Instructions

6. Clone Repository

git clone (https://github.com/kondamahesh1250/Military-Asset-Management.git)

cd military-asset-management

7. Backend Setup

cd backend

npm install

Create .env file:

MONGO_URI=mongodb://127.0.0.1:27017/militaryDB

JWT_SECRET=secret

Run server:

---

node index.js

3. Frontend Setup

---

cd frontend

npm install

npm start

🔑 Test Credentials

[

{ "username": "admin", "password": "123", "role": "ADMIN" },

{ "username": "commander", "password": "123", "role": "COMMANDER" },

{ "username": "logistics", "password": "123", "role": "LOGISTICS" }

]

🔁 API Endpoints

Auth

POST /api/auth/login

Assets

GET /api/assets

POST /api/assets

Transfers

GET /api/transfers

POST /api/transfers

Assignments

GET /api/assignments

POST /api/assignments

🧠 How It Works

User logs in → receives JWT token

Token stored in frontend

Each request is validated using middleware

Role determines access to features

Data is stored and updated in MongoDB

🎯 Key Highlights

Dynamic dropdowns (asset & base filtering)

Real-time stock validation

Prevents invalid transfers

Secure role-based API access

Clean and scalable architecture

💡 Future Enhancements

📊 Charts & analytics dashboard

🔁 Undo/rollback operations

🔔 Notifications for low stock

🌐 Deployment (Render + Vercel)